ADD / CHANGE / REMOVE ROWS

DML syntax

```
INSERT INTO t (c1,c2) VALUES (v1,v2),(v3,v4);
INSERT INTO t (c1,c2) SELECT a,b FROM s WHERE ..; -- list counts mat
UPDATE t SET c1=v1, c2=v2 WHERE cond;
DELETE FROM t WHERE cond;
UPDATE t SET c=s.c FROM s WHERE t.k=s.k;  -- from other table
DELETE FROM t USING s WHERE t.k=s.k;      -- delete via other
```

- **UPDATE:** WHERE picks rows first; SET right side uses **old** values. SET col has **no alias** (`SET salary`), **no column list** after table.
- Omitted INSERT cols get DEFAULT, else NULL.
- DELETE allows **no JOIN** — use USING / subquery. Don't repeat target in UPDATE..FROM.

PICK ROWS VIA ANOTHER TABLE

IN / EXISTS + traps

```
-- rows that HAVE a match
UPDATE Employees SET salary=salary*2
WHERE dept_id IN (SELECT dept_id FROM Departments
                  WHERE dept_name ILIKE '%research%');
-- EXISTS (keyword + correlation line!)
UPDATE Employees e SET salary=salary*2
WHERE EXISTS (SELECT 1 FROM Departments d
   WHERE d.dept_id=e.dept_id AND d.dept_name ILIKE '%sales%');
-- rows with NONE
DELETE FROM Departments d
WHERE d.budget=0.00
  AND NOT EXISTS (SELECT 1 FROM Employees e
                  WHERE e.dept_id=d.dept_id);
```

- **ILIKE**=case-insensitive. `%`=any run, `_`=1 char; no % ⇒ whole string.
- **NOT IN with a NULL ⇒ nothing: use NOT EXISTS. `=NULL` never true: use IS NULL. Scalar subquery = 1 row. No WHERE = every row.**

CHANGE STRUCTURE (ROWS KEPT)

ALTER TABLE (DDL)

```
ADD COLUMN c TYPE [NOT NULL] [DEFAULT v];
DROP COLUMN c [CASCADE];
RENAME COLUMN old TO new;   RENAME TO newtable;
ALTER COLUMN c TYPE T [USING c::T];
ALTER COLUMN c SET DEFAULT v;  / DROP DEFAULT
ALTER COLUMN c SET NOT NULL;   / DROP NOT NULL
ADD CONSTRAINT n CHECK(..) | UNIQUE(c)
  | PRIMARY KEY(c) | FOREIGN KEY(c) REFERENCES p(k);
DROP CONSTRAINT n;
```

- Each line is `ALTER TABLE t <action>;`. Several actions = comma-sep; RENAME can't combine.
- NOT NULL/DEFAULT use **ALTER COLUMN**, not ADD CONSTRAINT. Change a constraint = DROP then ADD.
- **Fails: SET NOT NULL w/ NULLs; ADD CONSTRAINT w/ bad rows; TYPE won't fit (USING); ADD COLUMN NOT NULL on filled table w/o DEFAULT; DROP COLUMN used by a view (CASCADE).**

PARENT DELETED/UPDATED

Foreign-key actions

Option	Child rows
`NO ACTION` (default)	reject if children exist
`CASCADE`	deleted too / get new key
`SET NULL`	key col → NULL (not if in child PK)
`SET DEFAULT`	key col → its default

- Change an action: DROP then ADD the FK. Default name `table_col_fkey`.
- **Circular FK** (dept_id ↔ dept_head): one col nullable, add FK later via ALTER; or DEFERRABLE + SET CONSTRAINTS DEFERRED.

RECIPE

Rows via another table

- Write inner `SELECT key FROM other WHERE..`; run it alone in your head.
- Outer: `WHERE key IN (inner)` (has match) or `NOT EXISTS(..WHERE o.key=t.key)` (none).
- EXISTS needs the correlation line. SET no alias; UPDATE no col list; DELETE no JOIN.

Queries & PL/pgSQL

joins · GROUP BY · HAVING · functions · exceptions

COMBINE & SUMMARIZE

Joins · GROUP BY · HAVING

- Order: SELECT · FROM · **WHERE**(rows) · GROUP BY · **HAVING**(groups) · ORDER BY. WHERE can't use aggregates.
- INNER = matches only. LEFT = all left, right NULL if none. Self-join = 2 aliases (`e.manager_id=m.emp_id`).
- Every non-aggregate SELECT col must be in GROUP BY. Qualify shared names.
- LEFT no-match: `COUNT(*)`=1 (padded row), `COUNT(right.col)`=0. HAVING on AVG/SUM drops empty LEFT groups — use COALESCE to keep.

TEMPLATES

Aggregate queries

```
-- per-dept stats, only avg > 5000
SELECT d.dept_name, COUNT(e.emp_id) AS employees_no,
       MAX(e.salary) AS max_salary
FROM Departments d
JOIN Employees e ON e.dept_id=d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 5000.00;
-- keep empty depts
LEFT JOIN Employees e ON e.dept_id=d.dept_id
.. COALESCE(SUM(e.salary),0.00) ..
-- above own-dept average
WHERE e.salary > (SELECT AVG(e2.salary) FROM Employees e2
                  WHERE e2.dept_id=e.dept_id);
-- payroll over budget
GROUP BY d.dept_id,d.dept_name,d.budget
HAVING SUM(e.salary) > d.budget;
```

PROCEDURAL CODE

PL/pgSQL blocks & functions

```
DO $b$
DECLARE _t INTEGER := 0;
BEGIN
  FOR k IN 1..10 LOOP _t := _t + k; END LOOP;
  IF _t > 50 THEN RAISE INFO 'big %', _t;
  ELSIF _t = 0 THEN RAISE INFO 'none';
  ELSE RAISE INFO 'small %', _t; END IF;
END; $b$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION public.f_Name(_p TYPE, _o TYPE=dflt)
RETURNS TYPE AS $fb$
DECLARE _v TYPE := init;
BEGIN ... RETURN value; END;          -- every path returns
$fb$ LANGUAGE plpgsql;
DROP FUNCTION IF EXISTS public.f_Name(VARCHAR(20)); -- by arg types
```

- `:=` assigns; `;` ends each stmt; tags match. `ELSIF`, `END IF;`. **EXIT** leaves a loop, **RETURN** ends the function.
- Defaults go at the end. Prefix vars with `_`. `SELECT c INTO _v..` = NULL if no row; MAX/MIN over 0 rows = NULL, COUNT = 0.

VALIDATE, DON'T CRASH

Exceptions & sentinels

```
IF _emp_id IS NULL
   OR NOT EXISTS(SELECT 1 FROM Employees WHERE emp_id=_emp_id)
THEN RETURN -1.00; END IF;              -- bad id / none
BEGIN                                   -- risky cast, own block
  _salary := TRIM(_new_salary)::DECIMAL(10,2);
EXCEPTION
  WHEN invalid_text_representation THEN RETURN -2.00; -- '12abc',''
  WHEN numeric_value_out_of_range  THEN RETURN -3.00; -- too big
END;
IF _salary < 0 THEN RETURN -4.00; END IF;  -- bad value
UPDATE Employees SET salary=_salary WHERE emp_id=_emp_id;
IF NOT FOUND THEN RETURN -5.00; END IF;     -- no row updated
RETURN _salary;
```

- Check order: **arguments → existence → value**. Codes negative (real salary is positive); comment each code.
- Failed cast aborts the stmt — wrap in its own `BEGIN..EXCEPTION..END;`. `NUMERIC(10,2)` max 99999999.99.

1-BASED · DATE PARTS

Arrays, dates, strings

fn	does
`a || b`	concat text
`LOWER/UPPER/LENGTH`	case / length
`TRIM/LTRIM/RTRIM`	strip both/left/right
`POSITION('x' IN s)`	1-based, 0=not found
`LEFT/RIGHT(s,n)`	first / last n
`SUBSTRING(s FROM 4 FOR 8)`	slice
`ASCII/CHR`	'R'=82; digits '0'-'9'=48-57

- Arrays **1-based**; `CARDINALITY`=len (NULL→NULL). Test `x = ANY(arr)`. EXTRACT(MONTH/DAY/DOY FROM d). Fixed dates only (no Thanksgiving).

```
_h INTEGER[] := ARRAY[101,704,1111,1225]; -- mo*100+day
_k := EXTRACT(MONTH FROM _date)*100 + EXTRACT(DAY FROM _date);
RETURN _k = ANY(_h);   -- or TO_CHAR(d,'MM-DD') = ANY(txt[])
```

STORED QUERY

Views

```
CREATE VIEW v (c1,c2) AS SELECT ...;   DROP VIEW v;
-- employee info: self-join for manager
CREATE OR REPLACE VIEW Employee_Info AS
SELECT e.emp_id, e.fname||' '||e.lname AS full_name,
       d.dept_name, e.salary,
       COALESCE(m.fname||' '||m.lname,'N/A') AS manager
FROM Employees e
JOIN Departments d ON d.dept_id=e.dept_id
LEFT JOIN Employees m ON m.emp_id=e.manager_id;
```

- Rows computed on use; dropping leaves base tables. LEFT JOIN keeps top manager / no-dept rows.
- **Updatable** (SQL-92): 1 base table, select+project only, no aggregates. Insert → NULL in hidden cols (fails if PK/NOT NULL). WITH CHECK OPTION rejects rows leaving the condition.
- A view using a col blocks DROP COLUMN (RESTRICT) until dropped.

DEPT INFO FUNCTION

Check order in a function

```
IF p_dept_id IS NULL THEN RETURN 'ID cannot be NULL'; END IF;
SELECT COUNT(*) INTO v_exists FROM Departments
 WHERE dept_id=p_dept_id;
IF v_exists=0 THEN RETURN 'does not exist'; END IF;  -- existence
SELECT COUNT(*), COALESCE(AVG(salary),0.00)
  INTO v_cnt, v_avg FROM Employees WHERE dept_id=p_dept_id;
IF v_cnt=0 THEN RETURN '..has 0 employees'; END IF;  -- empty
RETURN '..has '||v_cnt||' employees; avg '||v_avg;
```

- Separate branches: NULL arg vs. missing dept vs. empty dept. COALESCE guards head name / avg.

CONSTRAINTS VS STRUCTURES

Keys & indexes

- **Superkey** ⊇ key; **candidate** minimal; **primary** chosen. Keys=constraints; index=performance (search key need not be unique).
- **Clustered**: data order = index order; ≤1 per table; range = few contiguous pages. **Secondary/unclustered**: ~1 page read per match (can lose to scan). **Index-only**: answered from entries (ok unclustered).
- **Hash**: equality on **full** key only. **B+ tree**: equality + range on a **key prefix** — `<age,sal>` can't do `sal>40`.
- Composite index bigger; updated when any field changes. SQL std has no CREATE INDEX. Entry forms: record / `<key,rid>` / `<key,rid-list>`.

SEARCH COST

B+ tree

```
Root: 13 | 17 | 24 | 30
[2 3 5 7]->[14 16]->[19 20 22]->[24 27 29]->[33 34 38 39]
Search 15: 13<=15<17 -> leaf [14 16] -> absent.
Range 17..27: leaf for 17, follow 'next' links.
cost = root->leaf path + leaves read. height ~3-4.
```

ALL OR NOTHING

Transactions & ACID

- **A**tomic (all/none) · **C**onsistent in→out · **I**solation (= some serial order) · **D**urable (survives crash).

Isolation	Dirty	Unrep.	Phantom
READ UNCOMMITTED	maybe	maybe	maybe
READ COMMITTED	no	maybe	maybe
REPEATABLE READ	no	no	maybe
SERIALIZABLE	no	no	no

- Conflicts (same obj, ≥1 write): WR=dirty read, RW=unrepeatable, WW=overwrite. Phantom=new matching row between reads.
- **Strict 2PL**: S lock to read, X to write; hold all to commit/abort. Deadlock=wait cycle, DBMS aborts one.
- COMMIT / ROLLBACK / SAVEPOINT s / ROLLBACK TO SAVEPOINT s.

TWO UPDATES AS ONE

Atomic transaction

```
DO $B$
DECLARE _ret INTEGER;
BEGIN
  UPDATE Departments SET dept_head=2317 WHERE dept_id=19
    RETURNING dept_id INTO _ret;
  IF _ret IS NULL THEN RAISE EXCEPTION 'Dept not found'; END IF;
  _ret := NULL;
  UPDATE Employees SET dept_id=19 WHERE emp_id=2317
    RETURNING emp_id INTO _ret;
  IF _ret IS NULL THEN RAISE EXCEPTION 'Emp not found'; END IF;
END; $B$ LANGUAGE plpgsql;
-- plain: BEGIN; UPDATE..; UPDATE..; COMMIT;  (or ROLLBACK;)
```

- A DO block / function is one unit — any error or RAISE EXCEPTION rolls back all. Validate with RETURNING..INTO + NULL check. PREPARE holds 1 stmt only.

E-C-A · COLUMN SECURITY

Triggers & permissions

- **Trigger** = Event + Condition + Action; before / after / instead of; per row / per statement. Proc params: IN, OUT, INOUT.

```
-- hide salary: column grants
REVOKE SELECT ON Employees FROM PUBLIC;
GRANT SELECT(emp_id,fname,lname,dept_id,manager_id)
      ON Employees TO PUBLIC;
GRANT SELECT(salary) ON Employees TO salary_readers;
-- or hide behind a view without the column
CREATE VIEW Employee_Public AS
  SELECT emp_id,fname,lname,dept_id,manager_id FROM Employees;
GRANT SELECT ON Employee_Public TO PUBLIC;
```

SCAN THE NIGHT BEFORE

Common mistakes → fixes

Wrong	Right
`UPDATE t (col) SET`	`UPDATE t SET` (no col list)
`SET e.salary=`	`SET salary=` (no alias)
`WHERE (SELECT 1..)`	`WHERE EXISTS(SELECT 1..)`
EXISTS, no correlation	add `AND d.id=e.id`
subq+GROUP BY `=0`	drop GROUP BY / NOT EXISTS
`ALTER t ALTER col`	`ALTER TABLE t ALTER COLUMN`
`LIKE 'research'`	`ILIKE '%research%'`
`a + b` (strings)	`a || b`
`col = NULL`	`col IS NULL`
`ELSEIF` / no END IF	`ELSIF` / `END IF;`
array index from 0	arrays start at **1**
aggregate in WHERE	move to HAVING
RETURN to leave a loop	`EXIT` (RETURN ends fn)

PICK THE TOOL

Which construct?

Asked for	Use
change structure	ALTER TABLE
change row values	UPDATE
rows match / none	IN / NOT EXISTS
keep unmatched left	LEFT JOIN
hide cols / save query	VIEW
validate + codes	function + EXCEPTION
all-or-none	transaction / function
auto reaction	trigger
fast lookup	index (hash=eq, tree=range)